

Aircraft-Agnostic Modular Framework (AAMF)

MATLAB/Simulink-based modular aircraft simulation and analysis framework supporting:

- Fixed-wing aircraft
 - Multicopter systems
 - Distributed propulsion configurations
 - Real-time simulation
 - CesiumJS visualization
 - UDP/WebSocket communication
 - Joystick and HIL-style interfaces
-

Overview

The Aircraft-Agnostic Modular Framework (AAMF) is a unified flight simulation framework developed using MATLAB, Simulink, and Python.

The framework is built around a centralized `Aircraft` object that aggregates:

- Geometry
- Mass properties
- Aerodynamics
- Propulsion systems
- Control surfaces
- State representation
- Control vectors

Each subsystem independently computes force and moment contributions which are combined through a unified force and moment aggregation pipeline.

The same aircraft definition is reused for:

- Trim analysis
- Stability analysis
- Performance analysis
- Mission planning
- Time-domain simulation

- Real-time visualization
-

Repository Structure

```
AircraftDesign_Flightsim/
|
├─ MATLAB/
|   └─ classes/
|       Core object-oriented framework classes
|   └─ examples/
|       Example aircraft setup scripts
|   └─ Tutorials/
|       Tutorial examples
|   └─ Testcase/
|       Validation and testing scripts
|
├─ CesiumJS/
|   └─ main.py
|       Python WebSocket server
|   └─ index.html
|       CesiumJS visualization page
|
├─ UDP_Joystick_Stuff/
|   └─ Send_FDM_Torg.py
|       UDP joystick interface
|
└─ README.md
```

Framework Features

- Aircraft-agnostic architecture
- Modular object-oriented design
- Unified trim/performance/stability workflow
- Distributed propulsion support
- Simulink integration

- Real-time simulation capability
 - CesiumJS visualization
 - UDP/WebSocket communication
 - Joystick/HIL support
 - Reusable subsystem interfaces
-

Software Requirements

MATLAB

Recommended:

- MATLAB R2023a or newer
- Simulink
- Aerospace Blockset (recommended)

Python

Required packages:

- pygame
- websockets

Install using:

```
pip install pygame websockets
```

MATLAB Framework Setup

1. Open MATLAB

Navigate to:

```
AircraftDesign_Flightsim/MATLAB
```

2. Add Framework to MATLAB Path

Run:

```
addpath(genpath(pwd))  
savepath
```

3. Run Example Scripts

Example scripts are located in:

```
examples/
```

Example:

```
run('examples/C172_Trim_Example.m')
```

CesiumJS Visualization Setup

Windows

1. Navigate to CesiumJS Folder

```
cd "C:\Path\To\AircraftDesign_Flightsim\CesiumJS"
```

2. Create Python Virtual Environment

```
python -m venv venv
```

3. Activate Virtual Environment

```
venv\Scripts\activate
```

4. Install Required Package

```
pip install websockets
```

5. Run Web Server

```
python main.py
```

Expected output:

```
Running Webserver at http://localhost:8000
```

6. Open Browser

Open:

```
http://localhost:8000
```

Linux

1. Navigate to CesiumJS Folder

```
cd ~/AircraftDesign_Flightsim/CesiumJS
```

2. Create Python Virtual Environment

```
python3 -m venv venv
```

3. Activate Virtual Environment

```
source venv/bin/activate
```

4. Install Required Package

```
pip install websockets
```

5. Run Web Server

```
python3 main.py
```

6. Open Browser

Open:

```
http://localhost:8000
```

Joystick Interface Setup

Windows

1. Navigate to UDP Joystick Folder

```
cd "C:\Path\To\AircraftDesign_Flightsim\UDP_Joystick_Stuff"
```

2. Create Virtual Environment

```
python -m venv venv
```

3. Activate Environment

```
venv\Scripts\activate
```

4. Install Packages

```
pip install pygame websockets
```

5. Run Joystick Sender

```
python Send_FDM_Torg.py
```

Notes

Windows joystick handling uses:

```
pygame.joystick.Joystick(index)
```

Linux-specific interfaces such as:

```
/dev/input/js1  
evdev
```

are not used on Windows.

Linux

1. Navigate to UDP Joystick Folder

```
cd ~/AircraftDesign_Flightsim/UDP_Joystick_Stuff
```

2. Create Virtual Environment

```
python3 -m venv venv
```

3. Activate Environment

```
source venv/bin/activate
```

4. Install Packages

```
pip install pygame evdev
```

5. Check Joystick Devices

```
ls -l /dev/input/js*
```

Example:

```
/dev/input/js0  
/dev/input/js1  
/dev/input/js2
```

6. Run UDP Sender

```
python3 Send_FDM_Torg.py
```

Real-Time Simulation Workflow

Typical workflow:

1. Run MATLAB/Simulink aircraft model
2. Run joystick UDP sender
3. Run CesiumJS WebSocket server
4. Open browser visualization
5. Aircraft state is streamed in real time

Data pipeline:

```
MATLAB/Simulink  
  ↓  
UDP  
  ↓  
Python UDP Interface  
  ↓  
WebSocket
```



CesiumJS

Example Supported Aircraft

The framework has been tested using:

- Cessna 172
- F-16
- Boeing 737 (DATCOM-based)
- Quadrotor

The same framework architecture is reused across all aircraft configurations without modifying the aircraft-level equations of motion.

Citation

Naman Kumar Shetty
Aircraft-Agnostic Modular Framework (AAMF)
Virginia Polytechnic Institute and State University
Master of Science Report
2026

Author

Developed by:

Naman Kumar Shetty

Master of Science in Aerospace Engineering
Virginia Polytechnic Institute and State University
Blacksburg, Virginia

GitHub Repository:

<https://github.com/Idopt-Lab/aircraft-flight-simulator>